



Optimisation de code Code final Polymorphisme

Sommaire

Optimisation

[Code final](#)

[Édition de liens](#)

[Langages à objets](#)

Optimisation

Il peut y avoir une différence si
plusieurs comportements sont admis
par les règles du langage

Exemple : ordre d'évaluation des
expressions

Gain de temps :

$$\Sigma (\text{Exéc}_{\text{opt}} - \text{Exéc}_0) + \text{Compil}_{\text{opt}} - \text{Compil}_0$$

Améliorer le code intermédiaire pour que le code
final soit plus efficace

Sécurité (prioritaire) : ne pas changer le résultat
produit pour une entrée donnée

L'effet d'une modification peut être une
amélioration pour certains programmes (ou
certaines entrées) et non pour d'autres

On ne sait jamais si le code obtenu est optimal

L'optimisation sur les boucles est la plus efficace

La compilation optimisante prend plus de temps

Compilation à la volée (*just-in-time compilation*)

Optimisation **dynamique** du bytecode, pendant l'exécution

Utilise des statistiques sur les parties du bytecode les plus souvent exécutées

Compilation à l'avance (*ahead-of-time compilation*)

Utilise uniquement une analyse **statique** du code, sans l'exécuter

B1: $i := m - \$1$
 $j := n$
 $t1 := \$4 * n$
 $v := a[t1]$
B2: $i := i + \$1$
 $t2 := \$4 * i$
 $t3 := a[t2]$
 si $t3 < v$ aller à B2
B3: $j := j - \$1$
 $t4 := \$4 * j$
 $t5 := a[t4]$
 si $t5 > v$ aller à B3
B4: si $i \geq j$ aller à B6
B5: ...
B6: ...

Blocs de base (*basic blocks*)

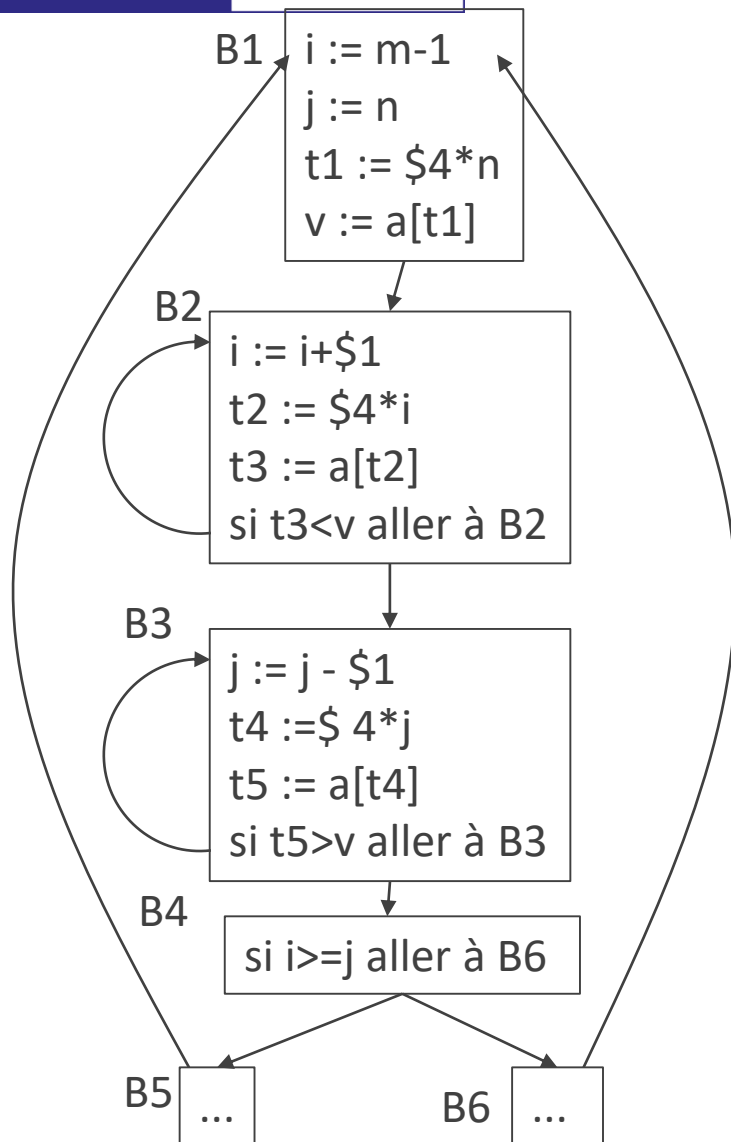
Objectif

Simplifier les raisonnements sur les séquences chronologiques qui mènent d'une instruction à une autre

Définition

Suite d'instructions

- sans branchement entrant (sauf vers la première)
- ni sortant (sauf depuis la dernière)



Graphe de flot de contrôle (*control-flow graph*)

Découper le code intermédiaire

- juste avant toute cible de saut
- juste après toute instruction de saut

Un bloc de base est toujours exécuté du début à la fin (sauf erreur à l'exécution)

Optimisation locale

Sur chaque bloc de base séparément

Optimisation globale

Sur plusieurs blocs de base en même temps

Exercice : faire de même sur write.asm à partir de print_begin

Élimination des sous-expressions communes

B5

```
t6 := $4*i
x := a[t6]
t7 := $4*i
t8 := $4*j
t9 := a[t8]
a[t7] := t9
t10 := $4*j
a[t10] := x
aller à B2
```

B5

```
t6 := $4*i
x := a[t6]
t8 := $4*j
t9 := a[t8]
a[t6] := t9
a[t8] := x
aller à B2
```

L'expression est calculée plusieurs fois

Les variables utilisées dans le calcul ne sont pas redéfinies entre deux calculs

Les variables à remplacer ne sont pas redéfinies entre le calcul et leur utilisation

Les deux calculs peuvent être dans deux blocs de base différents

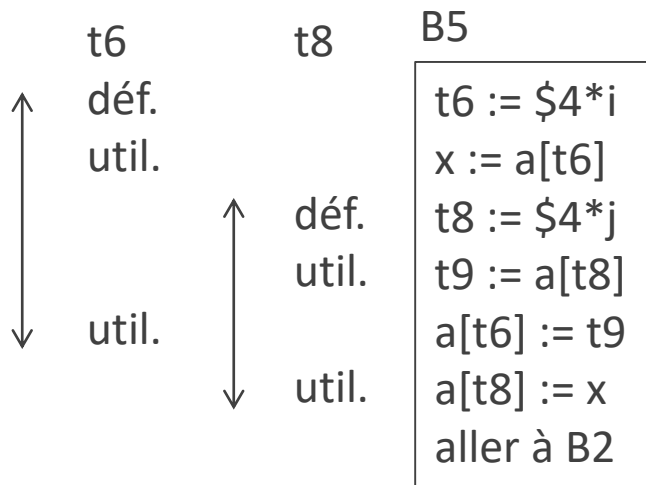
Élimination des sous- expressions communes

```
t1 := $4*i  
t2 := a[t1]  
b[x] := t2  
x := x+$1  
t3 := $4*i  
t4 := a[t3]  
b[x] := t4  
aller à B2
```

```
t1 := $4*i  
t2 := a[t1]  
b[x] := t2  
x := x+$1  
t4 := a[t1]  
b[x] := t4  
aller à B2
```

```
t1 := $4*i  
t2 := a[t1]  
b[x] := t2  
x := x+$1  
b[x] := t2  
aller à B2
```


Définitions et utilisations d'une variable



Vie (*liveness*) d'une variable

Entre une définition et la dernière utilisation correspondante

En dehors de cette période, la variable est morte (*dead store*)

La définition et les utilisations peuvent être dans deux blocs de base différents

Une variable peut avoir plusieurs vies, une à chaque affectation

Analyse à faire avant l'élimination des sous-expressions communes

Définitions et utilisations d'une variable

```
1  t1 := $4*i
2  t2 := a[t1]
3  b[x] := t2
4  x := x+$1
5  t3 := $4*i
6  t4 := a[t3]
7  b[x] := t4
8  aller à B2
```

Vie de
i
x
t3
t4

```
1  t := a-b
2  u := a-c
3  v := t+u
4  d := v+u
```

Vie de
t
u
v

« Propagation des copies » (*copy propagation*)

B5

```
x := t3  
a[t2] := t5  
a[t4] := x  
aller à B2
```

B5

```
x := t3  
a[t2] := t5  
a[t4] := t3  
aller à B2
```

```
debug = FALSE ;  
(...)  
if (debug)  
    printf("%d\n", node) ;
```

Après une instruction de copie $u := v$, utiliser
l'original plutôt que la copie tant que ni l'un ni
l'autre n'a été redéfini
Ça devrait s'appeler « propagation des originaux »
On augmente les chances d'éliminer u

Calcul des constantes (*constant folding*)

```
return c + ('a' - 'A')
```

```
return c + 'a' - 'A'
```

```
return 'a' - 'A' + c
```

Si une expression ne contient que des constantes,
la remplacer par une constante à la
compilation

Le calcul des expressions constantes peut produire
de nouvelles instructions de copie à propager

La propagation des copies peut produire de
nouvelles expressions constantes

Itération

```
return ('a' - 'A' + i) * ('a' - 'A' + j)
```

Calcul des constantes

```
t2 := $97
t3 := $65
t4 := t2 - t3
t5 := i + t4
t6 := $97
t7 := $65
t8 := t6 - t7
t9 := j + t8
t10 := t5 * t9
retour
```

```
t4 := $97 - $65
t5 := i + t4
t8 := $97 - $65
t9 := j + t8
t10 := t5 * t9
retour
```

```
t4 := $32
t5 := i + t4
t8 := $32
t9 := j + t8
t10 := t5 * t9
retour
```

```
t5 := i + $32
t9 := j + $32
t10 := t5 * t9
retour
```

Propagation de copie
Calcul de constante
Propagation de copie

Élimination du code inutile (*unreachable code*)

```
debug = FALSE ;  
(...)  
if (debug)  
    printf("%d\n", node) ;
```

```
debug = FALSE ;  
(...)  
if (FALSE)  
    printf("%d\n", node) ;
```

B5

```
x := t3  
a[t2] := t5  
a[t4] := t3  
aller à B2
```

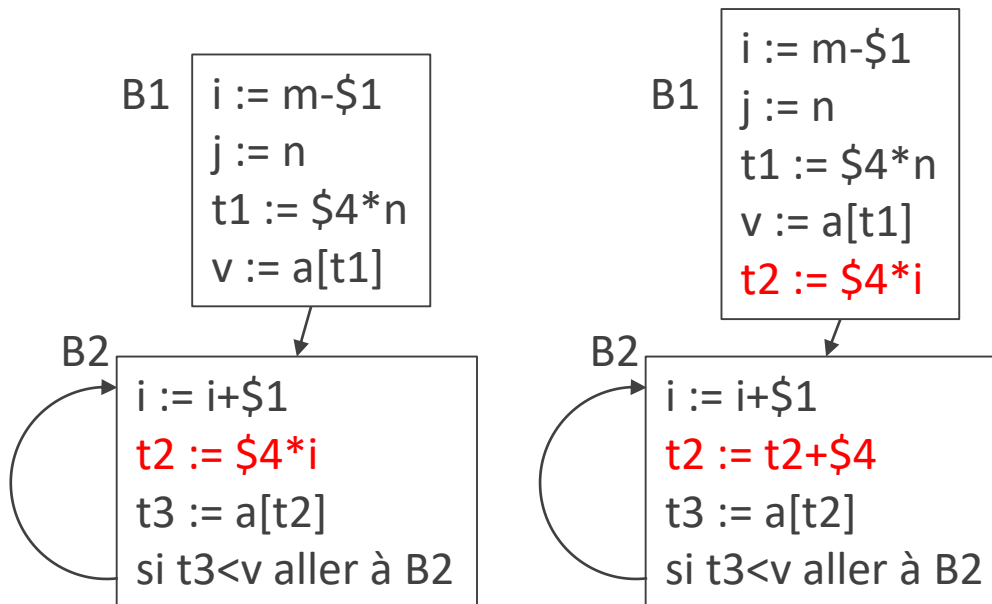
B5

```
a[t2] := t5  
a[t4] := t3  
aller à B2
```

Un bloc de base qui n'est accessible que par une condition toujours fausse est inutile

La définition d'une variable morte est inutile

Variables induites (*induction variables*)



t2 est lié à i

Le développeur a utilisé i

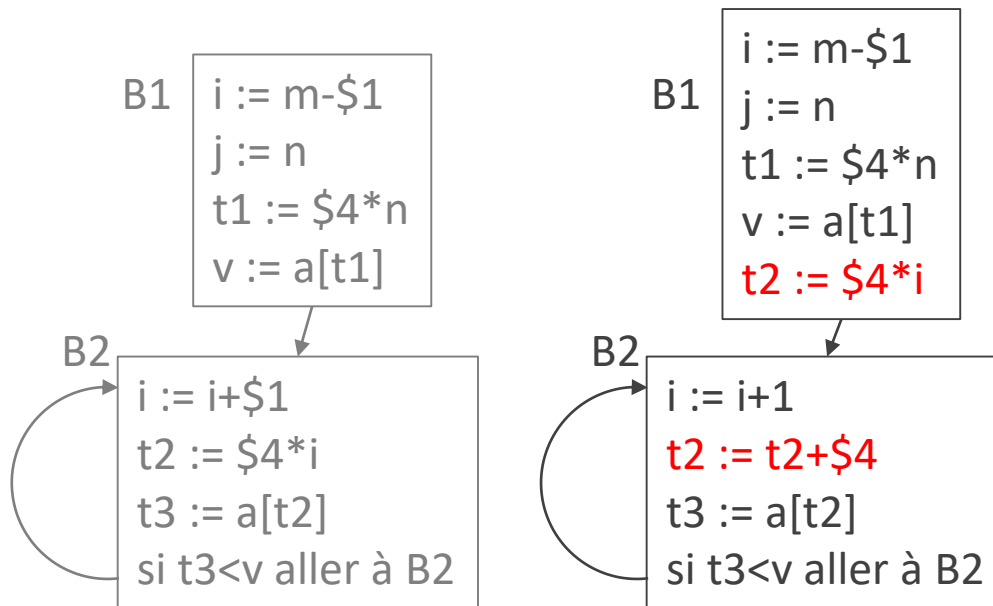
Le compilateur a introduit t2 :
variable induite

On n'a pas forcément besoin des
deux

**Réduction de force (*strength
reduction*)**

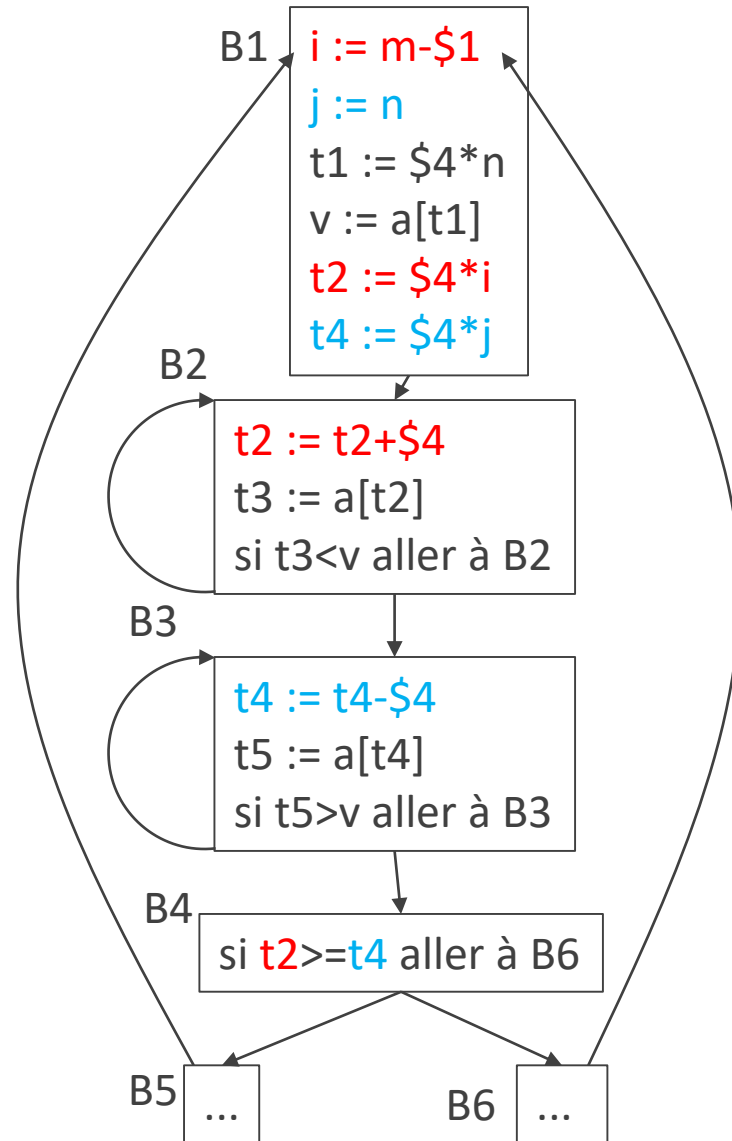
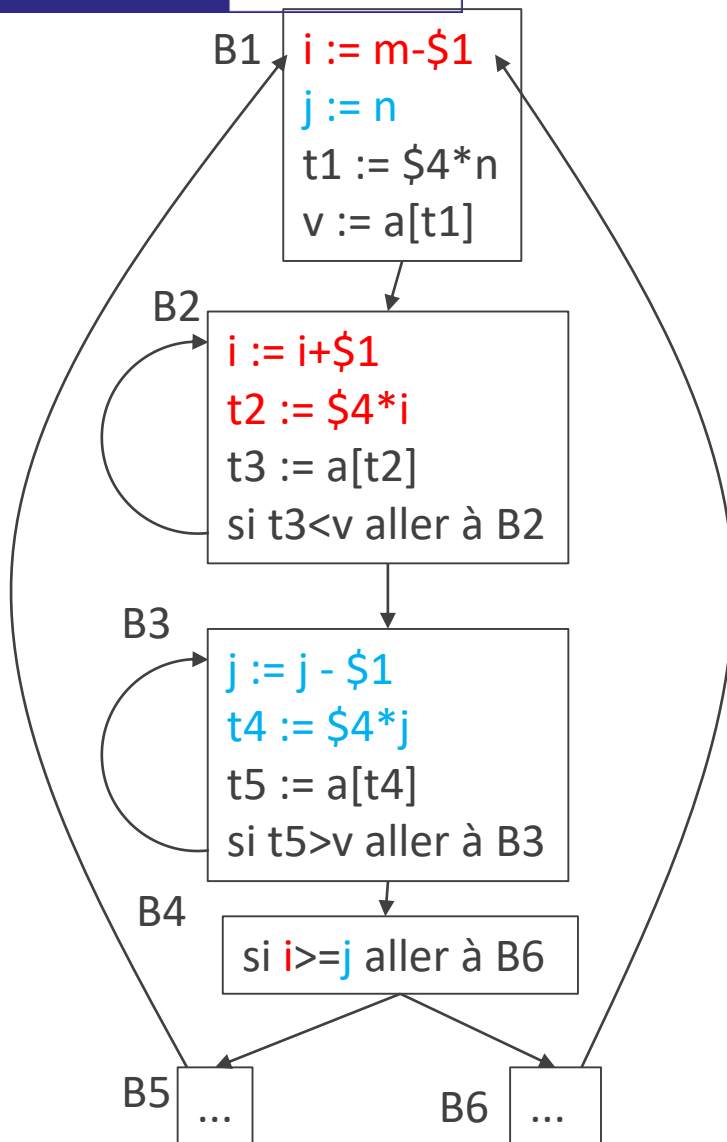
Remplacer une multiplication par
une addition

Variables induites



Si i n'est plus utilisé après B2, on peut l'éliminer de B2

Variables induites



Déplacement de code (*code motion*)

```
while (i <= limit-2)  
    a[i] += j ;
```

```
t = limit-2 ;  
while (i <= t)  
    a[i] += j ;
```

```
while (i <= strlen(a))  
    a[i] += j ;
```

```
t = strlen(a) ;  
while (i <= t)  
    a[i] += j ;
```

Si un calcul dans une boucle donne le même résultat à toutes les itérations (invariant de boucle), le déplacer à l'extérieur de la boucle

Optimisation interprocédurale

Nécessite d'analyser plusieurs fonctions à la fois
À l'édition de liens

Simplifications algébriques

$x := \$0 * y$

$x := \$0$

$x := \$1 * y$

$x := y$

Réduction de force

$x := \$2 * y$

$x := y + y$

$x := y ** \$2$

$x := y * y$

Sommaire

Optimisation

Code final

Édition de liens

Langages à objets

Code final

mov dest, src
add dest, src
sub dest, src

Machine à registres

Environ 16 à 256 registres $R_0, R_1, \dots, R_n$

Mémoire adressable par octets ou par mots

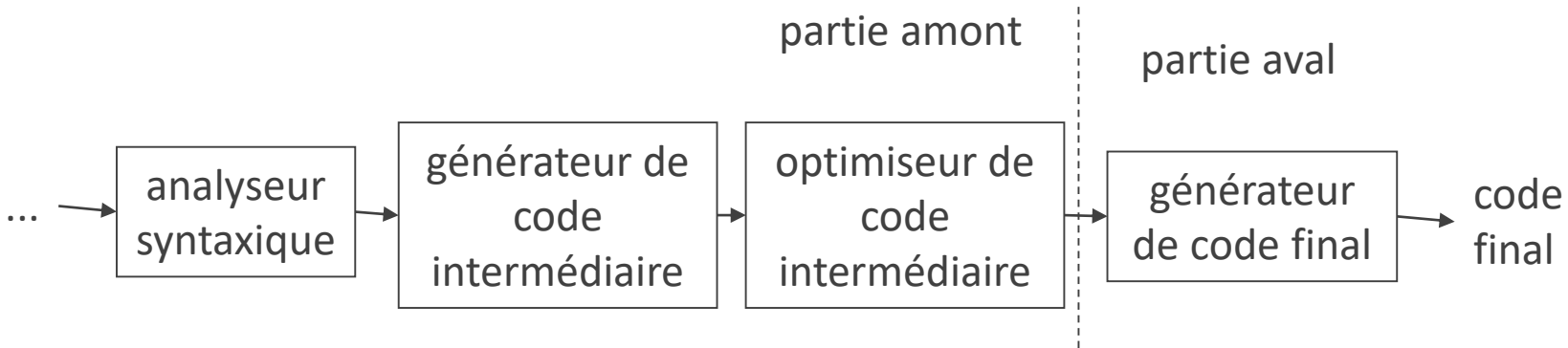
Instructions

Environ 8 à 256 instructions

Opérations : deux adresses

sub dest, src dest := dest – src

Génération de code final



Code final

Exemple (presque final) : assembleur

Dépendant de l'architecture de la machine cible

Séparer code intermédiaire et final permet de

- réutiliser un générateur de code intermédiaire sur des machines différentes
- réutiliser un optimiseur de code intermédiaire

Exemples de différences entre architectures cibles

```
; .nasm
mov  rax, 4
sub  [b], rax
```

```
; .nasm
push qword [b]
push qword 4
pop  rax
sub  qword [rsp], rax ; b-4
pop  qword [b]
```

```
; .nasm
mov  rax, qword [b]
sub  rax, 4
mov  qword [b], rax
```

Nombre de registres

Environ 16 à 256

Instructions

CISC : certains opérandes peuvent être en mémoire

RISC : tous les opérandes sont dans des registres sauf pour les instructions d'accès mémoire

Utilisation de la pile ou de registres

Pour évaluer une expression, sauvegarder les opérandes

- soit dans la pile
- soit dans des registres tant qu'il y en a

Code final pour $a := (b - 4) * c$

; with registers

```
mov  eax, dword [b]
sub   eax, 4
mov  ebx, dword [c]
imul  eax, ebx
mov  rbx, a
mov  dword [rbx], eax
```

; with stack

```
push a
movsx rax, dword [b]
push rax
push 4
pop  rbx
pop  rax
sub  rax, rbx
push rax
```

movsx rbx, dword [c]

```
push rbx
pop  rbx
pop  rax
imul rax, rbx
push rax
pop  rbx
pop  rax
mov  dword [rax], ebx
```

Suivant les architectures, la solution la plus efficace utilise la pile ou les registres

Différences entre code intermédiaire et code final

$t_1 := \$12 \text{ div } t_2$ `mov  $R_1$ , 12`
 `idiv  $R_2$`

Certaines instructions nécessitent un registre de travail

`if  $t_2 = \$0$  goto 328` `cmp  $R_2$ , 0`
 `je 328`

Les sauts peuvent nécessiter deux instructions

$t_2 := \$8 \text{ mul } t_1$
 $t_3 := c[t_2]$ `mov  $R_1$ , [c+8* $R_0$ ]`

Le contraire existe aussi : modes d'adressage

- **LFB** pour *label function*
begin
- **ebp** pour *base pointer*
- **esp** pour *stack pointer*
- **pushl** pour *push long*
- etc.

```
#include <stdio.h>
```

```
int main(void) {
    float radius=6.4,circum;
    circum=6.28*radius;
    return 0;
}
```

```
.file "arith.c"
.def __main; .scl 2; .type 32; .endef
.text
.globl _main
.def __main; .scl 2; .type 32; .endef
__main:
LFB7:
    .cfi_startproc
    pushl %ebp
    .cfi_def_cfa_offset 8
    .cfi_offset 5, -8
    movl %esp, %ebp
    .cfi_def_cfa_register 5
    andl $-16, %esp
    subl $32, %esp
    call __main
    movl $LC0, %eax
    movl %eax, 28(%esp)
    flds 28(%esp)
    fldl LC1
    fmulp %st, %st(1)
    fstps 24(%esp)
    movl $0, %eax
    leave
    .cfi_restore 5
    .cfi_def_cfa 4, 4
    ret
    .cfi_endproc

LFE7:
    .section .rdata,"dr"
    .align 4
LC0:
    .long 1087163597
    .align 8
LC1:
    .long 1374389535
    .long 1075388088
    .ident "GCC: (GNU) 4.9.3"
```

Modes d'adressage

Registre	R_0
Adresse en mémoire	$[a]$
Constante	a

Plusieurs opérations à la fois

Modèle simplifié d'assembleur

`mov  $[a]$ ,  $R_0$`

`add  $R_0$ , 4`

`sub  $[b]$ ,  $[R_0]$`

`mul  $[b]$ ,  $[c+R_0]$`

Adresse donnée par un registre

$[R_0]$

Adresse relative par rapport à une base
donnée par un registre $[a+R_0]$

Adresse contenue en mémoire à une adresse
relative $[[a+R_0]]$

Taille des instructions

mov [a], R_0	2
add R_0 , 4	2
sub [b], [R_0]	2
mul [b], [$c+R_0$]	3

Les instructions n'occupent pas toutes la même
taille mémoire

Chaque instruction occupe au moins 1 mot
mémoire

1 mot additionnel pour chaque adresse

1 mot additionnel pour chaque constante

Modèle simplifié : le cout d'exécution d'une
instruction est représenté par sa taille
mémoire

$a := b + c$

```
mov R0, [b]
add R0, [c]      6
mov [a], R0
```

```
mov [a], [b]      6
add [a], [c]
```

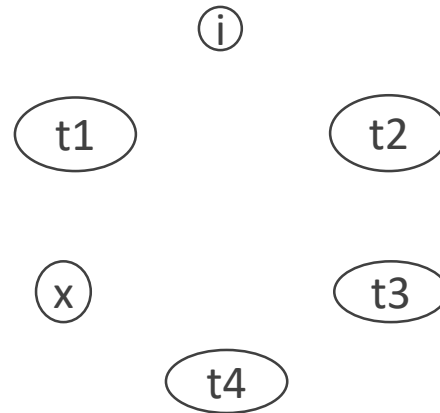
```
mov R0, R1      2
add R0, R2
```

```
mov [R0], [R1]  2
add [R0], [R2]
```

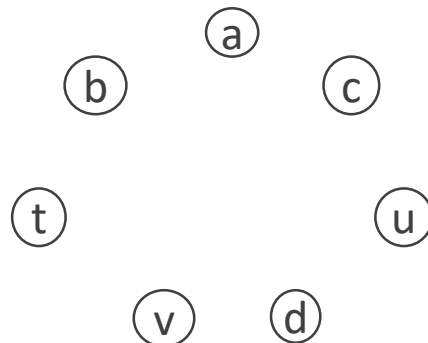
Dans la 3^e traduction, on suppose que a, b et c
sont rangés dans R_0 , R_1 et R_2

Dans la 4^e traduction, on suppose que les adresses
de a, b et c sont dans R_0 , R_1 et R_2

```
t1 := 4*i
t2 := d[t1]
e[x] := t2
x := x+1
t3 := 4*i
t4 := d[t3]
e[x] := t4
aller à B2
```



```
t := a-b
u := a-c
v := t+u
d := v+u
```



Choix des registres

On ne peut pas mettre deux variables dans le même registre si elles **interfèrent** (une des deux est définie pendant que l'autre est vivante)

Relier deux variables par un arc si elles interfèrent

On suppose

- que a, b, i et x sont définis avant
- que d et e sont les seules variables utilisées après

Choix des registres

```
t := a-b
u := a-c
v := t+u
d := v+u
e := d
```

R_0	d, e
R_1	u
R_2	u

a	a
b	b
c	c
d	R_0
e	R_0
t	
u	R_1, R_2
v	

Deux variables peuvent être
rangées dans le même registre

Deux registres peuvent contenir la
même variable

Un algorithme de choix des registres

Une variable peut être rangée dans un registre le temps de faire un calcul

Pour chaque instruction $x := y \text{ op } z$

- choisir où mettre x (L)
- puis choisir où trouver y (y')
- éventuellement écrire **mov** L, y'
- puis choisir où trouver z (z')
- écrire **op** L, z'

Un algorithme de choix des registres

var_descriptor

a	a
b	b
c	c
d	
t	
u	
v	

reg_descriptor

R_0
R_1

Choix prioritaire de cet algorithme : mettre le résultat de l'instruction dans un registre qui contient déjà le premier opérande

Choix 2 : dans un registre inutilisé

Choix 3 : libérer un registre déjà pris

Choix 4 : dans la variable de destination

Deux tableaux

Pour chaque variable, à quelles adresses et dans quels registres elle est rangée

Pour chaque registre, quelles variables il contient

$t := a - b$

$u := a - c$

$v := t + u$

$d := v + u$

$t := a - b$

$\text{mov } R_0, [a]$

$\text{sub } R_0, [b]$

var_descriptor		reg_descriptor	
a	a	R_0	t
b	b	R_1	
c	c		
d			
t	R_0		
u			
v			

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)

$L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i

$L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)

choisir y' dans var_descriptor[y], de préférence un registre
émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre

émettre (op L, z')

si y n'est plus utilisé ensuite

retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite

retirer z de reg_descriptor[]

mov R_0 , [a]
sub R_0 , [b]

t := a-b

u := a-c

v := t+u

d := v+u

u := a-c mov R_1 , [a]
sub R_1 , [c]

var_descriptor		reg_descriptor	
a	a	R_0	t
b	b	R_1	u
c	c		
d			
t	R_0		
u	R_1		
v			

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)
L = R_i

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i
L = R_i

sinon L = x

si ($L \notin \text{var_descriptor}[y]$)
choisir y' dans var_descriptor[y], de préférence un registre
émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre
émettre (op L, z')

si y n'est plus utilisé ensuite
retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite
retirer z de reg_descriptor[]

```

t := a-b
u := a-c
v := t+u
d := v+u

```

```

v := t+u      add R0, R1

```

var_descriptor		reg_descriptor	
a	a	R_0	t
b	b	R_1	u
c	c		
d			
t	R_0		
u	R_1		
v			

```

mov R0, [a]
sub R0, [b]
mov R1, [a]
sub R1, [c]

```

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)
 $L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i
 $L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)
 choisir y' dans var_descriptor[y], de préférence un registre
 émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre
 émettre (op L, z')

si y n'est plus utilisé ensuite

retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite

retirer z de reg_descriptor[]

```

t := a-b
u := a-c
v := t+u
d := v+u

```

```

v := t+u      add R0, R1

```

var_descriptor		reg_descriptor	
a	a	R ₀	v
b	b	R ₁	u
c	c		
d			
t			
u	R ₁		
v	R ₀		

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)
 $L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i

$L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)

choisir y' dans var_descriptor[y], de préférence un registre
 émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre
 émettre (op L, z')

si y n'est plus utilisé ensuite

retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite

retirer z de reg_descriptor[]

```

t := a-b
u := a-c
v := t+u
d := v+u

```

```

mov R0, [a]
sub R0, [b]
mov R1, [a]
sub R1, [c]
add R0, R1

```

```

d := v+u
add R0, R1

```

var_descriptor		reg_descriptor	
a	a	R ₀	v
b	b	R ₁	u
c	c		
d			
t			
u	R ₁		
v	R ₀		

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)
 $L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i
 $L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)
 choisir y' dans var_descriptor[y], de préférence un registre
 émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre
 émettre (op L, z')

si y n'est plus utilisé ensuite
 retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite
 retirer z de reg_descriptor[]

```

t := a-b
u := a-c
v := t+u
d := v+u

```

```

mov R0, [a]
sub R0, [b]
mov R1, [a]
sub R1, [c]
add R0, R1

```

```

d := v+u
add R0, R1

```

var_descriptor		reg_descriptor	
a	a	R ₀	d
b	b	R ₁	
c	c		
d	R ₀		
t			
u			
v			

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)
 $L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i
 $L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)
 choisir y' dans var_descriptor[y], de préférence un registre
 émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre
 émettre (op L, z')

si y n'est plus utilisé ensuite
 retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite
 retirer z de reg_descriptor[]

t := a-b

u := a-c

v := t+u

d := v+u

mov R_0 , [a]

sub R_0 , [b]

mov R_1 , [a]

sub R_1 , [c]

add R_0 , R_1

add R_0 , R_1

mov [d], R_0

d := v+u

add R_0 , R_1

mov [d], R_0

var_descriptor	reg_descriptor
----------------	----------------

a	a	R_0	d
---	---	-------	---

b	b	R_1	
---	---	-------	--

c	c		
---	---	--	--

d	R_0 , d		
---	-----------	--	--

t			
---	--	--	--

u			
---	--	--	--

v			
---	--	--	--

pour chaque instruction $x := y \text{ op } z$

si (y est seul dans R_i et n'est plus utilisé ensuite) ou (reg_descriptor[R_i] est vide)

$L = R_i$

sinon si (x est utilisé ensuite) ou (op nécessite un registre)

choisir un registre R_i

$L = R_i$

sinon $L = x$

si ($L \notin \text{var_descriptor}[y]$)

choisir y' dans var_descriptor[y], de préférence un registre

émettre (mov L, y')

choisir z' dans var_descriptor[z], de préférence un registre

émettre (op L, z')

si y n'est plus utilisé ensuite

retirer y de reg_descriptor[]

si z n'est plus utilisé ensuite

retirer z de reg_descriptor[]

Code binaire

Opcode	Instruction
04 <i>ib</i>	ADD AL, <i>imm8</i>
05 <i>iw</i>	ADD AX, <i>imm16</i>
05 <i>id</i>	ADD EAX, <i>imm32</i>
REX.W + 05 <i>id</i>	ADD RAX, <i>imm32</i>
80 /0 <i>ib</i>	ADD r/m8, <i>imm8</i>

Codes numériques Noms symboliques

Le nom symbolique (*mnemonics*, ex. **add** ou **+**) des instructions est remplacé par un code numérique (*opcode*, ex. **04**)

Pour un même nom symbolique, une instruction a généralement plusieurs variantes suivant les propriétés des opérandes, et chacune est représentée par un code différent (*opcode*)

Sommaire

Optimisation

Code final

Édition de liens

Langages à objets

module A

table des points d'entrée
table des références externes

instructions machine
données statiques

module B

table des points d'entrée
table des références externes

instructions machine
données statiques

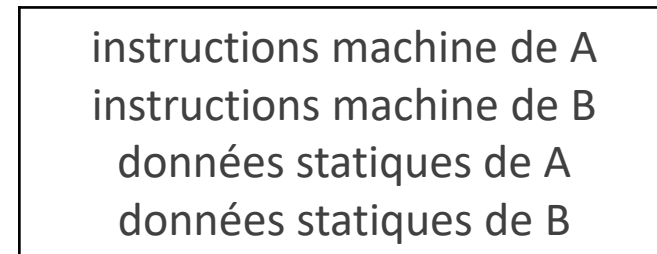
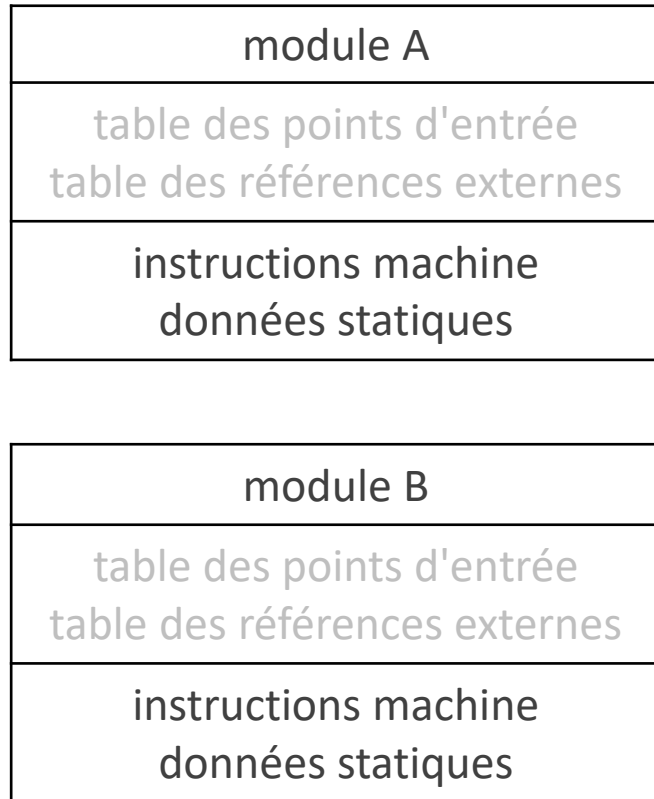
Édition de liens

Dans un module, il peut y avoir des appels à une fonction définie dans un autre module

Les bibliothèques de fonctions (*libraries*) sont comme des modules

On peut tout traduire, sauf l'adresse de la fonction appelée

Édition de liens



Translation d'adresses

Les adresses sont relatives à l'adresse où commence le segment dans le module
Pour chaque adresse dans un module compilé séparément, calculer de combien il faut la traduire dans le programme cible

Édition de liens

Chaque module compilé séparément contient des tables de noms

identification
table des points d'entrée table des références externes
instructions machine données statiques

Table des points d'entrée

Pour chaque nom utilisable dans les modules extérieurs,
son adresse dans le module courant

Table des références externes

Pour chaque référence à un nom défini dans un module extérieur,
l'adresse de la référence dans le code

Édition de liens

identification
table des points d'entrée table des références externes
instructions machine données statiques

Traitement des références externes d'un module **M**

Pour chaque référence à un nom défini dans un module extérieur **M'**,
trouver l'adresse du nom dans **M'** et la traduire
aller à la référence dans le code de **M**
remplacer la référence par l'adresse

Sommaire

Optimisation

Code final

Édition de liens

Langages à objets

Définitions

graphicalObject
colour
translate
scale

`image.scale(factor)`

`scale(image, factor)`

Objets

regroupent les structures de données et les opérations

Champs : données sur un objet

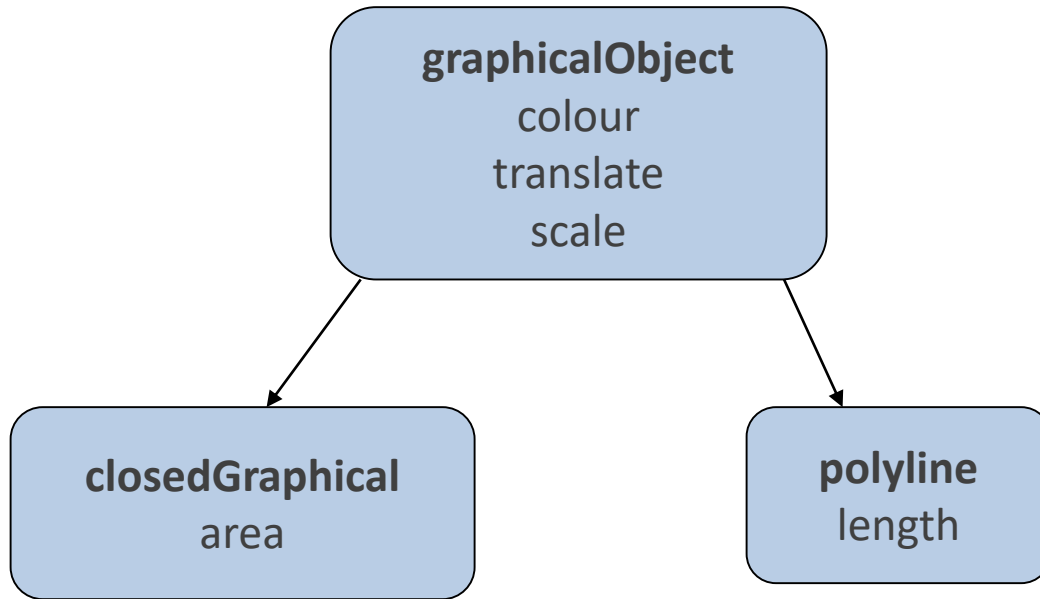
Méthodes : fonctions sur un objet

Classe : type d'objets

Traduction d'une méthode

Fonction qui a comme premier paramètre l'objet sur lequel on invoque la méthode

Héritage



Sous-classe ou classe dérivée : moins d'objets que la classe de base, plus d'informations

Les objets des classes dérivées ont les champs et méthodes de la classe de base, plus d'autres

Héritage

rectangle

Champs de la classe de base :
colour

Champs supplémentaires :
side1_length
side2_length

Accès aux champs

Les champs définis dans la classe de base
conservent la même adresse relative dans les
classes dérivées

Cette adresse est connue à la compilation

Réécriture de méthodes

On peut redéfinir une méthode lors d'un héritage

Exemple. Dans la classe **polyline**, la méthode

scale provoque un changement d'échelle.

```
class rectangle : public polyline {  
    double side1_length, side2_length ;  
}
```

Il faut redéfinir **scale** pour qu'elle change les
deux longueurs des côtés

Polymorphisme

Type statique

a := b

Le type statique de **a** est déduit de la déclaration de **a**

La copie de **b** sera vue comme ayant le type de **a**

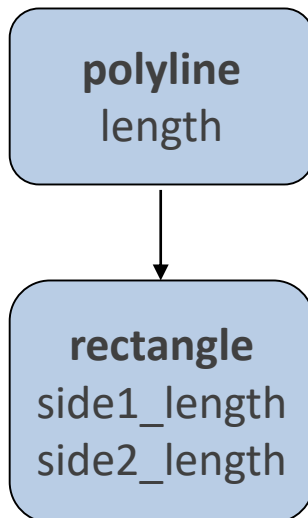
Les opérations applicables à **a** pourront être appliquées à la copie de **b**

poly := rect

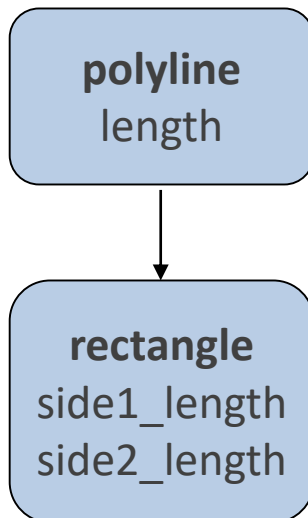
Un objet du type dérivé peut être vu comme ayant le type de base (on perd les informations spécifiques au type dérivé)

Les opérations applicables au type de base restent applicables au type dérivé

L'inverse n'est pas vrai



Polymorphisme



Type dynamique

a := **b**

Le type propre ou type dynamique de **a** peut être le type de **b**

Il est connu seulement à l'exécution

Certaines opérations dépendent du type propre (sélection dynamique de la méthode)

Polymorphisme

Paramètres des fonctions

```
rectangle dist(point a) ;  
// ...  
dist(b)    // b sera vu comme un point
```

Valeur de retour

```
rectangle dist(point a) {  
// ...  
return c ; }  
           // c sera vu comme un rectangle
```

Sélection des méthodes

Méthode statique

Sélection par le type statique

Méthode d'instance

Sélection dynamique

La sélection de la méthode à appeler dépend du type propre de l'objet, connu seulement à l'exécution

L'objet a un lien vers un descripteur de classe (statique)

Le descripteur de classe a les liens vers le code des méthodes

Sélection des méthodes

`graphicalObject`

<code>translate_GO</code>
<code>scale_GO</code>

`polyline`

<code>translate_PL</code>
<code>scale_PL</code>
<code>length_PL</code>

`rectangle`

<code>translate_PL</code>
<code>scale_RA</code>
<code>length_RA</code>

L'adresse relative des liens est la même pour la
définition de base et pour les redéfinitions

Le contenu du lien est l'adresse du code de la
méthode. Il dépend du type propre

Sélection des méthodes

Traduction d'un appel

`graphicalObject`

<code>translate_GO</code>
<code>scale_GO</code>

`rectangle`

<code>translate_PL</code>
<code>scale_RA</code>
<code>length_RA</code>

```
image.scale(factor)
```

Traduction :

```
vt=*image;
```

```
vm=vt[2];
```

```
(*vm)(image, factor) ;
```

Le compilateur relie les noms des méthodes virtuelles (**scale**) aux adresses relatives dans la table virtuelle (2)

Construction des tables virtuelles

`graphicalObject`

<code>translate_GO</code>
<code>scale_GO</code>

`polyline`

<code>translate_PL</code>
<code>scale_PL</code>
<code>length_PL</code>

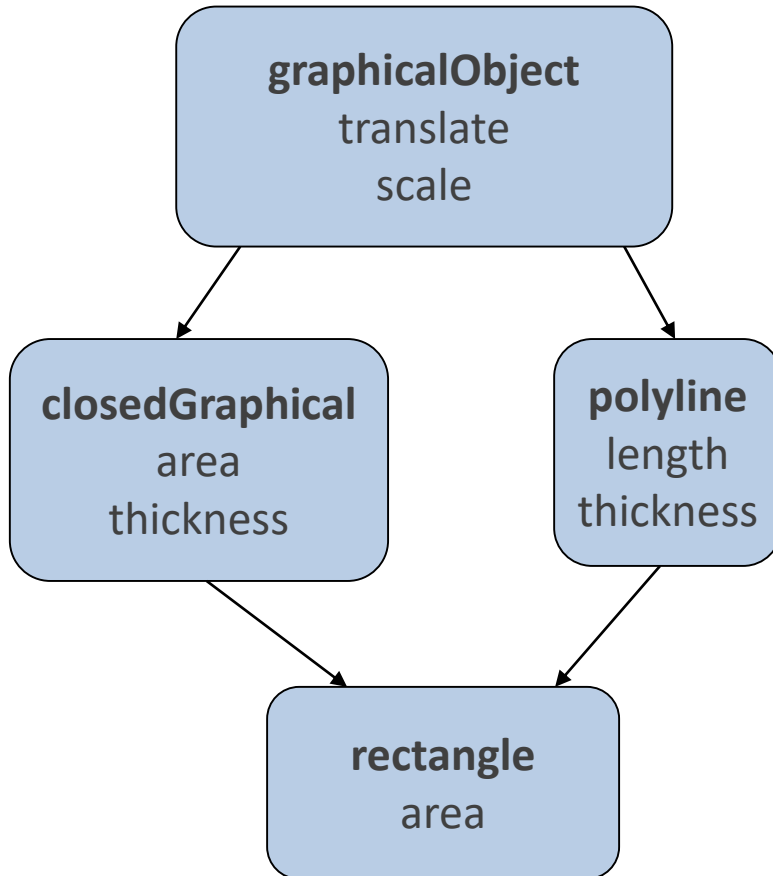
`rectangle`

<code>translate_PL</code>
<code>scale_RA</code>
<code>length_RA</code>

- On copie la table virtuelle de la classe de base
- On remplace les méthodes redéfinies par l'adresse du nouveau code
- On ajoute les nouvelles méthodes

Cout de la sélection dynamique

- pour chaque objet, le pointeur vers la table virtuelle
- pour chaque classe, la table virtuelle
- à chaque appel de méthode, 1 indirection + 1 accès dans la table



Héritage multiple

Une classe peut avoir plusieurs classes mères
La hiérarchie d'héritage est un graphe acyclique,
non un arbre

Conflit de noms si deux classes mères utilisent le
même nom pour des champs ou méthodes
Les conflits sont résolus par des méthodes propres
à chaque langage

En C++, préfixe de résolution de portée

`closedGraphical :: thickness`

`polyline :: thickness`

Héritage multiple

classe mère 1 : **closedGraphical**

classe mère 2 : **polyline**

attributs supplémentaires

Pour chaque classe mère, le compilateur doit pouvoir voir l'objet comme un objet de la classe mère

Les adresses relatives des attributs de **polyline** ne peuvent plus avoir la même adresse relative que dans **polyline**

```
myPolyline = myRectangle ;
```

le compilateur crée une vue **polyline** de **myRectangle**

Héritage répété

Une classe dérivée hérite d'une autre classe
plusieurs fois, directement ou indirectement

En C++, la classe mère est instanciée plusieurs fois
dans la classe dérivée (la même méthode
héritée plusieurs fois est utilisée pour des
objectifs différents)

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR